Chapter 5 worked solutions

Multiple Heads, Position, and the Complete Architecture

These written solutions cover Exercises 2 and 3. The companion notebook covers Exercises 1 and 4–12, including the two programming challenges. The calculations below follow the axis conventions and embedding scale used in the chapter.

2. Compare three attention shapes

Suppose we translate a batch of two sentences. Each source has five stored positions, and each shifted target input has three. An encoder head compares each source position with every source key. A decoder self-attention head compares each target position with target keys. A decoder cross-attention head starts with a target query and looks across source keys. Each comparison produces one number, which becomes a weight after softmax.

An attention map has four axes in this implementation. The first selects one of the two examples. The second selects one of the four heads within that example. The last two identify the query position and the key position. Write $A_{b,h,i,j}$ for the weight that query i gives key j in example b and head h.

Attention operation	Query positions i	Key positions j	Map shape
Encoder self-attention	5 source positions	5 source positions	$[2, 4, 5, 5]$
Decoder self-attention	3 target positions	3 target positions	$[2, 4, 3, 3]$
Decoder cross-attention	3 target positions	5 source positions	$[2, 4, 3, 5]$

To build the shapes, first pick a single example and head. The encoder head makes $5 \cdot 5 = 25$ comparisons. Its five rows follow the queries, and each row has five entries for the keys. The decoder self-attention head makes a 3×3 grid. The cross-attention head makes a 3×5 grid because the decoder supplies three queries while the encoder memory supplies five keys and five paired values. Stacking the four grids for the heads inside each of the two examples adds the leading axes $[2, 4]$. The last axis counts keys even in the rectangular cross-attention case.

For a fixed example, head, and query, softmax normalizes along the final axis over allowed keys. A mask can set individual weights to zero, while the stored map keeps its shape. For example, the causal target self-attention mask can allow target query 2 to use target keys 1 and 2 while suppressing key 3, using one-based position numbers. These permissions still occupy a row of length three. The other rows have their own permissions and softmax denominators. Nothing is normalized across heads or across different queries.

A triangle serves a purpose when query and key positions run along the **same target timeline**. A decoder query can then use its current and preceding target inputs. Applying the analogous rule $j \leq i$ to the rectangular cross-attention map would let the first target query read just source position 1 and let the third read just the first three source positions. It would prevent every target query in this example from reading source positions 4 and 5. The full source has already been encoded, and there is no requirement that the first translated word align with the first source word. In *the red car* $\rightarrow$ *el coche rojo*, for instance, the decoder can read the last source word *car* when predicting *coche* at the second target position.

In cross-attention, use the source key validity mask to exclude source padding and allow the rest of the encoded sentence for every target query. Its shape $[2, 1, 1, 5]$ broadcasts over the four heads and three target queries. The same source mask also broadcasts over the five encoder queries in encoder self-attention. The causal restriction on target keys belongs to the separate decoder self-attention operation.

3. Move a token

Start with the embedding $\mathbf{e} = (0, 1, 0, 2)$. Its four coordinates belong to the token. When the same token moves, look up the same embedding and add the positional vector of its new location. Here the position index starts at zero, and the trigonometric arguments are in radians.

Let p be the position index and r the coordinate-pair index. The sine and cosine within one pair use the same angle $p/10000^{2r/d_{model}}$. With width $d_{model} = 4$, there are two pairs:

Pair index r	Coordinates, counted from zero	Divisor $10000^{2r/4}$	Angle
0	0 and 1	$10000^0 = 1$	p
1	2 and 3	$10000^{1/2} = 100$	$p/100$

The resulting vector at position 2 is

$$\begin{aligned} PE_2 &= (\sin 2,\ \cos 2,\ \sin 0.02,\ \cos 0.02) \\ &\approx (0.909297,\ -0.416147, 0.019999, 0.999800). \end{aligned}$$

The original embedding scale is $\sqrt{d_{model}} = 2$, so the vector contributed by the embedding is $2\mathbf{e} = (0, 2, 0, 4)$. Add this vector to PE_2 coordinate by coordinate:

$$\begin{aligned} \mathbf{u}_2 &= (0, 2, 0, 4) + PE_2 \\ &= (\sin 2,\ 2 + \cos 2,\ \sin 0.02,\ 4 + \cos 0.02) \\ &\approx (0.909297, 1.583853, 0.019999, 4.999800). \end{aligned}$$

At position 1, the two angles are 1 and 0.01. Use the same $2\mathbf{e} = (0, 2, 0, 4)$ when computing the other input vector:

$$\begin{aligned} PE_1 &= (\sin 1,\ \cos 1,\ \sin 0.01,\ \cos 0.01) \\ &\approx (0.841471, 0.540302, 0.010000, 0.999950), \\ \mathbf{u}_1 &\approx (0.841471, 2.540302, 0.010000, 4.999950). \end{aligned}$$

At coordinate 1, counted from zero, the positional addition changes from $+\cos 1$ to $+\cos 2$. The shared embedding contribution to that coordinate is 2, so the sum falls from about 2.540302 to 1.583853. At coordinate 3, the angle changes by only 0.01 because that pair's divisor is 100. The cosine and the resulting sum therefore change by much less in this example.

Moving the token changes p, and hence the two angles and the four entries in the positional term. The token embedding, its multiplier of 2, the frequency divisors, and the final width of four are fixed. Subtracting the two input vectors cancels their shared embedding contribution and gives $\mathbf{u}_2 - \mathbf{u}_1 = PE_2 - PE_1$. These are the input vectors before input dropout. The division by $\sqrt{d_k}$ occurs later, on dot-product scores inside each attention head.